

\newpage

Part I: Language Fundamentals

\newpage

Introduction

Sage is a clean, indentation-based systems programming language built in C. It combines the readability of Python with the performance of compiled languages, offering multiple compilation backends, a compile-time safety system inspired by Rust, and a self-hosted compiler written in Sage itself.

Key Features

- **Python-like syntax** with indentation-based blocks and explicit end keywords
- **9 execution backends**: C codegen, LLVM IR, native x86-64/aarch64/rv64 assembly, bytecode VM, JIT, AOT, Kotlin/Android
- **Compile-time safety**: ownership tracking, borrow checker, lifetimes, Option types
- **Self-hosted compiler** written in Sage (lexer, parser, interpreter, codegen) with dispatch-table optimizations
- **Rich standard library**: 23+ modules including math, io, string, sys, thread, JSON, collections
- **Object-oriented programming** with classes, inheritance, structs, enums, and traits
- **Generators and async**: yield-based generators, async/await concurrency
- **Three GC modes**: tracing (concurrent mark-sweep), ARC (reference counting), ORC (optimized RC with cycle detection)
- **Kotlin/Android transpiler**: `--emit-kotlin` and `--compile-android` generate complete Gradle projects from a single `.sage` file
- **Developer tooling**: formatter, linter, type checker, LSP server, REPL
- **Bare-metal and OS development**: Multiboot2, UEFI, QEMU integration
- **GPU programming**: Vulkan and OpenGL backends (covered in separate guides)
- **Machine learning**: neural networks, model training (covered in separate guides)
- **Performance library** (`lib/perf.sage`): dispatch tables, signal singletons, flat env cache, shape constructors
- **SageMetal VM**: freestanding bytecode interpreter for bare-metal (no malloc, no libc, no OS)
- **Metal stdlib** (`lib/metal/`): serial, GPIO, IRQ, timer, MMIO for kernel/embedded development
- **Default hybrid runtime**: JIT profiling on hosted, AST on bare-metal, automatic selection
- **v4.0.0 updates**: OIS integration, $O(1)$ dictionary size, $O(N)$ unique checks (simple types), native array reversal, and binary exponentiation for repeating (linear output work).
- **327 interpreter tests**, 1623 self-hosted tests (2060+ total)

Quick Start

Build from source

```
git clone https://github.com/Night-Traders-Dev/SageLang.git
cd SageLang
make

# Run a program
./sage hello.sage

# Interactive REPL
./sage
```

A minimal Sage program:

```
print "Hello, World!"
```

A slightly larger program demonstrating functions and control flow:

```
proc fibbuzz(n):
  for i in range(1, n + 1):
    if i % 15 == 0:
      print "FizzBuzz"
    elif i % 3 == 0:
      print "Fizz"
    elif i % 5 == 0:
      print "Buzz"
    else:
      print i
  end
end

fibbuzz(20)
```

\newpage

Variables and Types

Sage supports `let` for variable declaration and `var` for mutable variables.

Variable Declaration

```
# Immutable binding
let name = "Alice"
let age = 30
let pi = 3.14159
let active = true
let nothing = nil

# Mutable variable
var counter = 0
counter = counter + 1
print counter    # 1
```

Variables declared with `let` cannot be reassigned. Use `var` when you need a variable whose value changes over time.

Built-in Types

Type	Example	Description
Number	42, 3.14, 0xFF, 0o755	Integer or floating-point
String	"hello"	Text with escape sequences
Boolean	true, false	Logical values
Nil	nil	Absence of value

Type	Example	Description
Array	[1, 2, 3]	Ordered, mutable collection
Dict	{"a": 1}	Key-value mapping
Tuple	(1, "x")	Immutable ordered pair
Bytes	bytes(10)	Binary-safe byte buffer
VM Program	(result of asm)	Compiled bytecode

Sage numbers are IEEE 754 double-precision floating point. Integer values are exact up to 2^{53} . Hex literals (0xFF) and octal literals (0o755) are supported as of v1.4.

Type Checking

The `type()` function returns the type of any value as a string:

```
print type(42)          # "number"
print type("hello")     # "string"
print type(True)        # "bool"
print type(None)        # "nil"
print type([1, 2])      # "array"
print type({"a": 1})     # "dict"
print type((1, 2))      # "tuple"
```

Type Conversion

```
# Number to string
print str(42)           # "42"
print str(3.14)         # "3.14"
print str(True)         # "true"
print str(None)         # "nil"

# String to number
print tonumber("123")   # 123
print tonumber("3.14")  # 3.14

# Truncate to integer
print int(3.7)          # 3
print int(-2.9)         # -2

# Character conversions
print chr(65)           # "A"
print ord("A")          # 65
print chr(10)           # newline character
```

\newpage

Operators

Arithmetic Operators

```
print 5 + 2      # 7   (addition)
print 5 - 2      # 3   (subtraction)
print 5 * 2      # 10  (multiplication)
print 5 / 2      # 2.5 (division)
print 5 % 2      # 1   (modulo, uses fmod for floats)
print -5         # -5  (negation)

# Operator precedence follows standard math rules
print 2 + 3 * 4   # 14 (not 20)
```

```
print (2 + 3) * 4      # 20
```

The % operator uses `fmod()` internally, so it preserves float semantics:

```
print 7.5 % 2.5      # 0
print 3.7 % 1.0      # 0.7
```

Comparison Operators

```
print 1 == 1      # true
print 1 != 2      # true
print 1 < 2       # true
print 2 > 1       # true
print 1 <= 1      # true
print 1 >= 1      # true
```

Comparison works structurally for arrays, dicts, and class instances. Two instances with the same fields and values compare as equal.

Logical Operators

```
print true and false  # false
print true or false   # true
print not true        # false
```

***Important:** In Sage, 0 is **truthy**. Only false and nil are falsy. Use explicit comparisons: `if x == 0:` rather than `if not x:`.*

Bitwise Operators

```
print 5 & 3      # 1   (AND)
print 5 | 3      # 7   (OR)
print 5 ^ 3      # 6   (XOR)
print ~5         # -6  (NOT)
print 1 << 4     # 16  (left shift)
print 16 >> 2    # 4   (right shift)
```

String Concatenation

Strings are concatenated with +:

```
let greeting = "Hello" + ", " + "World!"
print greeting      # Hello, World!

# Convert other types before concatenation
let msg = "Count: " + str(42)
print msg           # Count: 42
```

\newpage

Control Flow

If / Elif / Else

Conditional blocks use `if`, `elif`, `else`, and are terminated with `end`:

```
let x = 42

if x > 100:
  print "big"
elif x > 10:
```

```

    print "medium"
else:
    print "small"
end

```

The `elif` keyword supports unlimited branches. Sage processes them as chained if/else structures internally.

```

let grade = 85

if grade >= 90:
    print "A"
elif grade >= 80:
    print "B"
elif grade >= 70:
    print "C"
elif grade >= 60:
    print "D"
else:
    print "F"
end

```

While Loops

```

var i = 0
while i < 5:
    print i
    i = i + 1
end

```

For Loops

```

# Iterate over arrays
let fruits = ["apple", "banana", "cherry"]
for fruit in fruits:
    print fruit
end

# Range-based for loop
for i in range(5):
    print i    # 0, 1, 2, 3, 4
end

# Range with start and end
for i in range(2, 5):
    print i    # 2, 3, 4
end

```

Break and Continue

```

# Break exits the loop
var i = 0
while true:
    if i == 5:
        break
    end
    print i
    i = i + 1
end

# Continue skips to the next iteration
for x in range(10):
    if x % 2 == 0:
        continue
    end
    print x    # 1, 3, 5, 7, 9
end

```

```
end
```

Match / Case

Pattern matching with optional guards:

```
let color = "red"
match color:
  case "red":
    print "Stop"
  end
  case "yellow":
    print "Caution"
  end
  case "green":
    print "Go"
  end
  default:
    print "Unknown color"
  end
end
```

Match with guards using if:

```
let score = 85
match score:
  case score if score >= 90:
    print "Excellent"
  end
  case score if score >= 70:
    print "Good"
  end
  default:
    print "Needs improvement"
  end
end
```

\newpage

Functions

Functions are declared with `proc` and terminated with `end`.

Basic Functions

```
proc greet(name):
  print "Hello, " + name + "!"
end

greet("Alice")    # Hello, Alice!
```

Return Values

```
proc add(a, b):
  return a + b
end

let result = add(3, 4)
print result    # 7
```

Default Parameters

```

proc connect(host, port, ssl):
  print "Connecting to " + host + ":" + str(port)
end

proc greet_user(name, greeting):
  print greeting + ", " + name + "!"
end

greet_user("Alice", "Hello")    # Hello, Alice!
greet_user("Bob", "Hi")        # Hi, Bob!

```

Recursion

```

proc factorial(n):
  if n <= 1:
    return 1
  end
  return n * factorial(n - 1)
end

print factorial(5)    # 120

proc fibonacci(n):
  if n <= 1:
    return n
  end
  return fibonacci(n - 1) + fibonacci(n - 2)
end

print fibonacci(10)   # 55

```

Closures

Functions capture their lexical scope, enabling closures:

```

proc make_adder(n):
  proc add(x):
    return x + n
  end
  return add
end

let add5 = make_adder(5)
print add5(10)    # 15
print add5(20)    # 25

```

Stateful Closures

```

proc make_counter():
  var count = 0
  proc increment():
    count = count + 1
    return count
  end
  return increment
end

let counter = make_counter()
print counter()    # 1
print counter()    # 2
print counter()    # 3

```

First-Class Functions

Functions are values and can be passed as arguments:

```

proc apply(f, x):
  return f(x)
end

proc double(n):
  return n * 2
end

print apply(double, 5)    # 10

# Anonymous-style via closures
let square = proc(x):
  return x * x
end

print square(4)          # 16

# Inline anonymous proc expressions
print apply(proc(x): x * x end, 5)    # 25

\newpage

```

Strings

Sage strings support escape sequences and a rich set of built-in operations.

String Basics

```

let greeting = "Hello, World!"
print len(greeting)    # 13

# Character access by index
print greeting[0]      # "H"
print greeting[7]      # "W"

```

Escape Sequences

As of v1.4, Sage supports the following escape sequences in string literals:

Escape	Character
\n	Newline (LF)
\t	Tab
\r	Carriage return
\\	Backslash
\"	Double quote
\'	Single quote
\0	Null byte
\a	Bell
\b	Backspace
\f	Form feed
\v	Vertical tab
\xHH	Hex byte (e.g. \x41 = "A")

```

print "Line one\nLine two"
print "Column1\tColumn2"

```



```
print "She said \\\"hi\\\""
print "Hex: \x48\x65\x6C\x6C\x6F"      # "Hello"
```

String Functions

```
# Case conversion
print upper("hello")          # "HELLO"
print lower("HELLO")          # "hello"

# Trimming
print strip("  hello  ")      # "hello"

# Replacement
print replace("hello", "l", "r")  # "herro"

# Splitting and joining
let parts = split("a,b,c", ",")
print parts[0]                 # "a"
print join(parts, "-")         # "a-b-c"
```

String Searching

```
print startswith("hello", "hel")  # true
print endswith("hello", "llo")    # true
print contains("hello world", "world") # true
print indexof("hello", "ll")      # 2
```

Character Conversions

```
print chr(65)      # "A"
print chr(10)      # newline
print ord("A")     # 65
print ord("z")     # 122
```

\newpage

Data Structures

Arrays

Arrays are ordered, mutable collections:

```
let nums = [1, 2, 3, 4, 5]
print nums[0]      # 1
print len(nums)    # 5

# Push and pop
push(nums, 6)
print len(nums)    # 6
let last = pop(nums)
print last         # 6

# Both push() and append() work identically
append(nums, 7)
print len(nums)    # 6

# Slicing
let sub = slice(nums, 1, 3)
print sub          # [2, 3]

# Nested arrays
let matrix = [[1, 2], [3, 4], [5, 6]]
```

```

print matrix[1][0]      # 3

# Iteration
var total = 0
for n in nums:
    total = total + n
end
print total

```

Dictionaries

Dictionaries are key-value mappings with string keys:

```

let person = {"name": "Alice", "age": 30}
print person["name"]      # "Alice"

# Assignment
person["email"] = "alice@example.com"

# Check keys
print dict_has(person, "name")      # true
print dict_has(person, "phone")     # false

# Get keys and values
let keys = dict_keys(person)
let vals = dict_values(person)

# Delete a key
dict_delete(person, "age")

# Build incrementally (no multiline dict literals)
let config = {}
config["host"] = "localhost"
config["port"] = 8080
config["ssl"] = true

```

Tuples

Tuples are immutable ordered collections:

```

let point = (3, 7)
print point[0]      # 3
print point[1]      # 7
print len(point)    # 2

```

\newpage

Part II: Advanced Language Features

\newpage

Object-Oriented Programming

Classes

Classes are defined with `class` and terminated with `end`. The `init` method serves as the constructor, and `self` refers to the current instance:

```

class Person:
  proc init(self, name, age):

```

```

        self.name = name
        self.age = age
    end

    proc greet(self):
        return "Hi, I'm " + self.name
    end
end

let p = Person("Alice", 30)
print p.name      # Alice
print p.greet()   # Hi, I'm Alice

```

Inheritance

Classes can inherit from a parent class. Use `super` to call parent methods:

```

class Animal:
    proc init(self, name):
        self.name = name
    end

    proc speak(self):
        return "..."
    end
end

class Dog(Animal):
    proc init(self, name, breed):
        super.init(name)
        self.breed = breed
    end

    proc speak(self):
        return "Woof!"
    end
end

let d = Dog("Rex", "German Shepherd")
print d.name      # Rex
print d.breed     # German Shepherd
print d.speak()   # Woof!

```

Note that `super` auto-injects `self` as the first argument, but as of v4.1.5, passing it explicitly is also supported for compatibility: `super.init(args)` or `super.init(self, args)`.

Deep Inheritance

```

class A:
    proc init(self, x):
        self.x = x
    end
end

class B(A):
    proc init(self, x, y):
        super.init(x)
        self.y = y
    end
end

class C(B):
    proc init(self, x, y, z):
        super.init(x, y)
        self.z = z
    end
end

```

```

    end
end

let obj = C(1, 2, 3)
print obj.x    # 1
print obj.y    # 2
print obj.z    # 3

```

Arrow Operator

The `->` operator is an alias for `.`, allowing C-style member access:

```

class Vec3:
  proc init(self, x, y, z):
    self->x = x
    self->y = y
    self->z = z
  end

  proc mag_squared(self):
    return self->x * self->x + self->y * self->y + self->z * self->z
  end
end

let v = Vec3(3, 4, 0)
print v->x          # 3
print v->mag_squared() # 25

```

\newpage

Structs, Enums, and Traits

Structs

Structs define data types with typed fields:

```

struct Point:
  x: Int
  y: Int
end

struct Color:
  r: Int
  g: Int
  b: Int
end

```

Struct definitions create constructor-like classes. Fields are accessed with dot notation.

Enums

Enums define a fixed set of named variants:

```

enum Direction:
  North
  South
  East
  West
end

enum Color:
  Red
  Green

```

```
    Blue
end
```

Each variant is accessible as `Direction.North`, `Color.Red`, etc.

Traits

Traits define method signatures that classes can implement:

```
trait Printable:
  proc to_string(self) -> String
end

trait Comparable:
  proc compare(self, other) -> Int
end
```

Traits provide a form of interface specification. Classes implementing a trait should provide all the methods declared in the trait.

\newpage

Exception Handling

Exception handling uses `try`, `catch`, `finally`, and `raise`, with blocks terminated by `end`:

Basic Try/Catch

```
try:
  let result = 10 / 0
catch e:
  print "Error: " + e
end
```

Try/Catch/Finally

The `finally` block always executes, whether or not an exception was raised:

```
try:
  print "Opening resource"
  raise "Something went wrong"
catch e:
  print "Caught: " + e
finally:
  print "Cleanup complete"
end
```

Raising Exceptions

```
proc divide(a, b):
  if b == 0:
    raise "Division by zero"
  end
  return a / b
end

try:
  print divide(10, 0)
catch e:
  print e    # Division by zero
end
```

Nested Exception Handling

```
try:
  try:
    raise "inner error"
  catch e:
    print "Inner: " + e
  end
  print "Outer continues"
catch e:
  print "Should not reach here"
end
```

Custom Exception Objects

You can raise any value as an exception, including dicts for structured errors:

```
proc validate_age(age):
  if age < 0:
    let err = {}
    err["type"] = "ValidationError"
    err["message"] = "Age cannot be negative"
    err["value"] = age
    raise err
  end
  return age
end

try:
  validate_age(-5)
catch e:
  print "Error: " + str(e)
end
```

\newpage

Generators and Iterators

Generator Functions

A function containing `yield` becomes a generator. Use `next()` to advance it:

```
proc count_up(start, limit):
  var i = start
  while i < limit:
    yield i
    i = i + 1
  end
end

let gen = count_up(0, 5)
print next(gen) # 0
print next(gen) # 1
print next(gen) # 2
print next(gen) # 3
print next(gen) # 4
```

Yielding Values

`yield` suspends the function and returns a value to the caller. The function resumes from the point after `yield` when `next()` is called again:

```

proc fibonacci_gen():
  var a = 0
  var b = 1
  while true:
    yield a
    let temp = a
    a = b
    b = temp + b
  end
end

let fib = fibonacci_gen()
for i in range(10):
  print next(fib)
end
# Output: 0, 1, 1, 2, 3, 5, 8, 13, 21, 34

```

\newpage

Async and Concurrency

Async Procedures

Sage supports `async proc` for concurrent execution. Use `await` to wait for the result:

```

async proc fetch_data(url):
  # Simulated async work
  return "data from " + url
end

let result = await fetch_data("https://example.com")
print result

```

Threads

The `thread` module provides OS-level threading with mutexes:

```

import thread

var shared = 0
let mtx = thread.mutex()

proc worker(id):
  for i in range(1000):
    thread.lock(mtx)
    shared = shared + 1
    thread.unlock(mtx)
  end
end

let t1 = thread.spawn(worker, 1)
let t2 = thread.spawn(worker, 2)
thread.join(t1)
thread.join(t2)

print shared    # 2000

```

Thread Sleep

```

import thread

print "Starting..."
thread.sleep(1.5)    # Sleep for 1.5 seconds

```

```
print "Done!"
```

True Atomic Operations

Sage provides C-level atomic operations via `__atomic` compiler builtins. These are truly atomic — safe for concurrent access from multiple threads without locks.

```
let counter = atomic_new(0)      # Create atomic integer, initial value 0

# These are safe to call from multiple threads simultaneously:
atomic_add(counter, 1)           # Atomic increment
atomic_add(counter, 5)           # Atomic add
let val = atomic_load(counter)   # Atomic read (returns 6)

# Compare-and-swap (CAS):
let ok = atomic_cas(counter, 6, 10) # If counter == 6, set to 10
# ok == true, counter is now 10

# Atomic exchange:
let old = atomic_exchange(counter, 0) # Set to 0, return old value (10)
```

Semaphores

POSIX semaphores for controlling access to a finite number of resources:

```
let sem = sem_new(3)            # 3 permits available

sem_wait(sem)                   # Acquire permit (blocks if none available)
sem_wait(sem)                   # Acquire another
# ... do work with the resource ...
sem_post(sem)                   # Release permit

let ok = sem_trywait(sem) # Non-blocking acquire (returns true/false)
```

SMP and Multicore

CPU topology detection and core affinity:

```
# Detect hardware
let cores = cpu_count()          # Logical CPUs (includes hyperthreads)
let physical = cpu_physical_cores() # Physical cores only
let ht = cpu_has_hyperthreading() # true if SMT/HT detected

# Pin thread to specific core
thread_set_affinity(0)           # Pin to core 0
let core = thread_get_core()     # Which core am I on?
```

Multicore work distribution using `lib/os/smp.sage`:

```
import os.smp

smp.print_topology() # Print CPU info

# Run work on all cores in parallel
let results = smp.on_all_cores(proc(core_id):
    return core_id * core_id
)

# Distribute array work across cores
let items = range(10000)
let sums = smp.parallel_for_cores(items, proc(core_id, slice):
    let total = 0
    for x in slice:
        total = total + x
    return total
```


)

\newpage

Pattern Matching

The `match` statement provides pattern matching with guards.

Basic Matching

```
let status = 404

match status:
  case 200:
    print "OK"
  end
  case 301:
    print "Moved"
  end
  case 404:
    print "Not Found"
  end
  case 500:
    print "Server Error"
  end
  default:
    print "Unknown status: " + str(status)
  end
end
```

Guards

Add conditions to cases with `if`:

```
let value = 42

match value:
  case value if value < 0:
    print "Negative"
  end
  case value if value == 0:
    print "Zero"
  end
  case value if value > 0 and value < 100:
    print "Small positive"
  end
  default:
    print "Large"
  end
end
```

Matching Strings

```
proc http_method(method):
  match method:
    case "GET":
      return "Retrieving resource"
    end
    case "POST":
      return "Creating resource"
    end
    case "PUT":
      return "Updating resource"
```

```

        end
        case "DELETE":
            return "Deleting resource"
        end
        default:
            return "Unknown method"
        end
    end
end

print http_method("GET")    # Retrieving resource

\newpage

```

Defer Statements

The `defer` statement schedules a statement to execute when the current scope exits, regardless of how it exits (normal return or exception):

```

proc process_file(path):
    let data = "opened"
    defer print "Closing file"

    print "Processing: " + path
    # ... do work ...
    return data
end

process_file("test.txt")
# Output:
#   Processing: test.txt
#   Closing file

```

Deferred statements execute in LIFO (last-in, first-out) order:

```

proc example():
    defer print "First deferred"
    defer print "Second deferred"
    defer print "Third deferred"
    print "Main body"
end

example()
# Output:
#   Main body
#   Third deferred
#   Second deferred
#   First deferred

```

\newpage

Advanced Examples

High-Level: Web Server Mock

This example demonstrates using dicts, classes, and strings to simulate a small web routing system:

```

class Router:
    proc init(self):
        self.routes = {}
    end

```

```

proc add_route(self, path, handler):
    self.routes[path] = handler
end

proc handle(self, request):
    let path = request["path"]
    if dict_has(self.routes, path):
        let handler = self.routes[path]
        return handler(request)
    else:
        return {"status": 404, "body": "Not Found"}
    end
end

end

let app = Router()

app.add_route("/", proc(req):
    return {"status": 200, "body": "Welcome home!"}
end)

app.add_route("/api/hello", proc(req):
    let name = dict_has(req, "name") ? req["name"] : "Guest"
    return {"status": 200, "body": "Hello, " + name}
end)

let response = app.handle({"path": "/api/hello", "name": "Sage"})
print response["body"] # Hello, Sage

```

Low-Level: Custom Binary Packet

Demonstrates bytes, manual memory layout, and hashing:

```

import crypto.hash as hash

proc create_packet(payload_str):
    let payload = hash.string_to_bytes(payload_str)
    let size = len(payload)

    # Packet: [Header: 4][Size: 4][Payload: N][Checksum: 4]
    let buf = bytes(4 + 4 + size + 4)

    # Header "SAGE"
    buf[0] = 83; buf[1] = 65; buf[2] = 71; buf[3] = 69

    # Size (little-endian)
    buf[4] = size & 255
    buf[5] = (size >> 8) & 255
    buf[6] = (size >> 16) & 255
    buf[7] = (size >> 24) & 255

    # Payload
    for i in range(size):
        buf[8 + i] = payload[i]
    end

    # Simple XOR checksum
    var cs = 0
    for i in range(8 + size):
        cs = cs ^ buf[i]
    end
    buf[8 + size] = cs

    return buf
end

let p = create_packet("Hello")

```

```
print "Packet size: " + str(len(p))
print "Header char: " + chr(p[0]) # S
```

\newpage

Module System

Importing Modules

```
# Import entire module
import math
print math.abs(-42)      # 42
print math.sqrt(16)     # 4

# Import specific items
from strings import pad_left
print pad_left("42", 5, "0") # "00042"

# Import with alias
import arrays as arr
let nums = [1, 2, 3]

# Import with item alias
from math import sqrt as square_root
print square_root(25)    # 5
```

Creating Modules

Any .sage file is a module. Top-level definitions are exported automatically:

```
# mylib.sage
let VERSION = "1.0"

proc hello(name):
  return "Hello, " + name
end

proc add(a, b):
  return a + b
end

# main.sage
import mylib
print mylib.VERSION      # 1.0
print mylib.hello("World") # Hello, World
print mylib.add(2, 3)    # 5
```

Standard Library Modules

The following native modules are built into the interpreter:

Module	Description
math	Mathematical functions and constants
io	File I/O operations
string	String searching, manipulation
sys	System info, args, env, exit
thread	OS-level threading and mutexes

The following bundled Sage modules are in the `lib/` directory:

Module	Description
<code>arrays</code>	Functional array operations
<code>strings</code>	String formatting and transformation
<code>dicts</code>	Dictionary utilities
<code>iter</code>	Iterator combinators and generators
<code>stats</code>	Statistical functions
<code>utils</code>	General-purpose utilities
<code>assert</code>	Testing assertions
<code>math</code>	Extended math (Sage-level supplement)
<code>json</code>	Full JSON parser and serializer

OS and System Modules

Module	Description
<code>os.sync</code>	Synchronization primitives (Mutexes, Semaphores)
<code>os.errno</code>	POSIX error codes and <code>strerror</code> utility
<code>os.smp</code>	Multicore work distribution and topology
<code>os.cpio</code>	Initramfs CPIO archive handling
<code>os.tmpfs</code>	In-memory temporary filesystem
<code>os.alloc</code>	Bump, free-list, and bitmap page allocators
<code>os.vfs</code>	Virtual filesystem abstraction layer
<code>os.linux.sysfs</code>	Linux /sys filesystem interface

`os.errno`

The `os.errno` module provides POSIX-standard error constants and a `strerror(err)` utility for human-readable error messages.

```
import os.errno as errno

print errno.EOENT      # 2
print errno.strerror(2) # "No such file or directory"

# Networking error codes (v4.1.5+)
print errno.ECONNRESET # 104
print errno.EINPROGRESS # 115
```

`os.alloc`

Provides memory management for kernel heaps via multiple strategy-specific allocators.

Function	Description
<code>bump_create(base, size)</code>	Create a fast, reset-only bump allocator.
<code>freelist_create(base, size)</code>	Create an allocator supporting individual <code>free</code> .
<code>bitmap_create(base, n, sz)</code>	Create a page-oriented bitmap allocator.
<code>heap_stats(alloc)</code>	Get unified stats: <code>bytes_used</code> , <code>bytes_free</code> , <code>num_allocs</code> .
<code>stats(alloc)</code>	Alias for <code>heap_stats</code> .

os.linux.sysfs

Interface for probing and reading attributes from the Linux /sys filesystem.

```
import os.linux.sysfs as sysfs

if sysfs.device_exists("/sys/class/net/eth0"):
  let info = sysfs.get_net_device_info("eth0")
  print info["address"]
end

# Read raw attributes
let freq = sysfs.read_sysfs_int("/sys/devices/system/cpu/cpu0/cpufreq/scaling_cur_freq")
```

\newpage

Part III: Safety and Type System

\newpage

Type Annotations

Sage supports optional type annotations on function parameters and return types.

Parameter Types

```
proc add(a: Int, b: Int) -> Int:
  return a + b
end

proc greet(name: String) -> String:
  return "Hello, " + name
end

proc is_even(n: Int) -> Bool:
  return n % 2 == 0
end
```

Optional Types

The T? syntax marks a type as optional (may be nil):

```
proc find_user(id: Int) -> String?:
  if id == 1:
    return "Alice"
  end
  return nil
end

let user = find_user(99)
if user != nil:
  print user
end
```

Generic Type Parameters

Type annotations support generic parameters for collections and dotted names for types:

```
let fs: vfs.VFS = vfs.VFS(dev)

proc sum_list(nums: Array[Int]) -> Int:
```

```

    var total = 0
    for n in nums:
        total = total + n
    end
    return total
end

```

Type annotations are checked by the type checker (`sage check`) and the safety system but are not enforced at runtime by the interpreter.

\newpage

The Safety System

Sage v4.1.5 includes a compile-time safety system inspired by Rust. It provides ownership tracking, borrow checking, lifetime analysis, Option type enforcement, and fearless concurrency checks.

The safety system is a **static analysis pass** that runs after parsing and before code generation. It does not affect the interpreter.

Safety Modes

Mode	Flag	Description
Off	(default)	No safety checks
Annotated	--safety	Only check <code>@safe</code> functions
Strict	--strict-safety	Enforce everything globally

Ownership and Move Semantics

Variables own their data. When a value is assigned to another variable, ownership is transferred (moved), and the original variable becomes invalid:

```

# In strict safety mode:
let data = [1, 2, 3]
let other = data      # ownership moves to 'other'
# print data          # ERROR: use after move
print other           # OK: [1, 2, 3]

```

Borrow Checker

The borrow checker enforces two rules:

1. You can have **any number of immutable borrows** at the same time.
2. You can have **exactly one mutable borrow** at a time.
3. You cannot have both mutable and immutable borrows simultaneously.

```

# Immutable borrows are fine together
let x = 42
let a = x    # immutable borrow
let b = x    # immutable borrow -- OK

# Mutable borrow is exclusive
var y = 10
# Only one mutable reference at a time

```

Lifetimes

Lifetimes ensure references do not outlive the data they point to:

```
proc get_ref():
  let local = "hello"
  return local    # WARNING: reference may outlive local data
end
```

Option Types

In safe contexts, nil values must be handled explicitly using Option semantics:

```
proc find(items, target):
  for item in items:
    if item == target:
      return item    # Some(item)
    end
  end
  return nil          # None
end

let result = find([1, 2, 3], 2)
# In strict mode, must check before use:
if result != nil:
  print result
end
```

The safety library provides `Some()`, `None()`, `unwrap()`, and `map()` for working with Option types explicitly.

Fearless Concurrency: Send and Sync

The safety system tracks Send and Sync traits:

- **Send:** A type is Send if ownership can be transferred between threads.
- **Sync:** A type is Sync if it can be safely shared between threads.

In strict mode, passing a non-Send value to `thread.spawn` is an error.

Unsafe Blocks

Operations that bypass safety checks must be wrapped in `unsafe`:

```
unsafe:
  let ptr = mem_alloc(1024)
  mem_write(ptr, 0, "int", 42)
  let val = mem_read(ptr, 0, "int")
  mem_free(ptr)
end
```

Running the Safety Analyzer

```
# Analyze with default (annotated) mode
./sage --safety program.sage

# Analyze with strict mode
./sage --strict-safety program.sage

# Via the sage command
./sage safety program.sage
```

\newpage

Part IV: Standard Library

\newpage

Built-in Modules

Math Module

The math module provides trigonometric, logarithmic, and utility functions:

```
import math

# Trigonometry
print math.sin(math.pi / 2)    # 1.0
print math.cos(0)              # 1.0
print math.tan(math.pi / 4)    # ~1.0
print math.asin(1.0)           # ~1.5708
print math.acos(0.0)           # ~1.5708
print math.atan(1.0)           # ~0.7854
print math.atan2(1.0, 1.0)     # ~0.7854

# Powers and roots
print math.sqrt(16)            # 4
print math.pow(2, 10)          # 1024
print math.log(math.e)         # 1
print math.log10(1000)         # 3
print math.exp(1)              # ~2.71828

# Rounding
print math.floor(3.7)          # 3
print math.ceil(3.2)           # 4
print math.round(3.5)          # 4
print math.abs(-42)            # 42
print math.fmod(7, 3)          # 1

# Min/max/clamp
print math.min(3, 7)           # 3
print math.max(3, 7)           # 7
print math.clamp(150, 0, 100)  # 100

# Checks
print math.isnan(0.0 / 0.0)    # true
print math.isinf(1.0 / 0.0)    # true

# Constants
print math.pi                  # 3.14159265358979
print math.e                   # 2.71828182845905
print math.tau                 # 6.28318530717959
print math.inf                 # Infinity
print math.nan                 # NaN

# Utility
print math.pow_int(2, 10)      # 1024 (binary exponentiation, O(log n))

# Arithmetic Visualization
# Evaluates an expression string and shows step-by-step work.
# Backends: "sage" (default), "c", "asm"
# Controlled by CLI flag --math-work=grade,exec
math.printm("123 + 456", backend="sage", formats=["grade"])
# Output:
# 123
# + 456
# ----
```

```
# 579

# Constants
print math.pi      # 3.14159265358979
print math.e       # 2.71828182845905
print math.tau     # 6.28318530717959
print math.inf     # Infinity
print math.nan     # NaN

# Random
print math.random() # random float in [0, 1)
```

IO Module

The `io` module provides file system operations:

```
import io

# Read and write files
let content = io.readfile("data.txt")
io.writefile("output.txt", "Hello, World!")
io.appendfile("log.txt", "New entry\n")

# File system queries
print io.exists("data.txt")      # true/false
print io.isdir("/home")         # true
io.mkdir("new_dir")              # Creates directory (0755 secure permissions)
io.mkpath("a/b/c")              # Creates recursive path
print io.filesize("data.txt")    # file size in bytes

# Binary I/O
let bytes = io.readbytes("image.png") # Read as Bytes value
print bytes_len(bytes)             # number of bytes

let buf = bytes("Hello\n")
io.writebytes("output.bin", buf)    # Write Bytes (also accepts Array)
io.appendbytes("log.bin", buf)      # Append Bytes

# Directory listing
let files = io.listdir(".")
for f in files:
  print f
end

# Remove files
io.remove("temp.txt")
```

String Module

The `string` module provides advanced string operations:

```
import string

# Search
print string.find("hello world", "world") # 6
print string.rfind("hello hello", "hello") # 6
print string.startswith("hello", "hel")    # true
print string.endswith("hello", "llo")      # true
print string.contains("hello world", "world") # true

# Character access
print string.char_at("hello", 1)            # "e"
print string.ord("A")                      # 65
print string.chr(65)                       # "A"

# Manipulation
```

```

print string.repeat("ab", 3)      # "ababab"
print string.count("banana", "an") # 2
print string.substr("hello", 1, 3) # "ell"
print string.reverse("hello")     # "olleh"

```

Sys Module

The sys module provides system-level utilities:

```

import sys

# Command-line arguments
let args = sys.args()
for arg in args:
  print arg
end

# Environment
print sys.platform      # "linux", "darwin", or "windows"
print sys.version       # "3"
print sys.getenv("HOME") # /home/user

# Timing
let start = sys.clock()
# ... do work ...
let elapsed = sys.clock() - start
print "Took " + str(elapsed) + " seconds"

# Sleep
sys.sleep(0.5)    # sleep for 0.5 seconds

# Exit
sys.exit(0)

# Dynamic Call
# Calls a native function or class constructor with arguments.
let res = sys.call(math.sqrt, 16) # 4

# Shell Execution
# exec() returns exit code, shell_exec() returns output string.
let code = sys.exec("ls")
let output = sys.shell_exec("whoami")

```

Thread Module

The thread module provides OS-level concurrency:

```

import thread

# Spawn a thread
proc worker(id):
  print "Worker " + str(id) + " running"
end

let t = thread.spawn(worker, 1)
thread.join(t)

# Mutexes for synchronization
let mtx = thread.mutex()
thread.lock(mtx)
# ... critical section ...
thread.unlock(mtx)

# Sleep
thread.sleep(0.1)    # sleep for 100ms

```

Collection Libraries

Arrays Module

```
import arrays

let nums = [5, 3, 8, 1, 9, 2]

# Functional operations
let doubled = arrays.map(nums, proc(x): return x * 2 end)
let evens = arrays.filter(nums, proc(x): return x % 2 == 0 end)
let total = arrays.reduce(nums, 0, proc(acc, x): return acc + x end)

# Search
print arrays.contains(nums, 8)      # true
print arrays.index_of(nums, 8)     # 2

# Transform
let rev = arrays.reverse(nums)
let combined = arrays.concat([1, 2], [3, 4])
```

Strings Module

```
import strings

let text = " Hello, World! "
print strings.compact(text)         # "Hello, World!"
print strings.pad_left("42", 5, "0") # "00042"
print strings.pad_right("hi", 10, ".") # "hi....."
print strings.repeat("ab", 3)       # "ababab"
print strings.surround("hello", "[", "]") # "[hello]"
print strings.snake_case("HelloWorld") # "hello_world"
print strings.dash_case("helloWorld")  # "hello-world"
print strings.csv(["a", "b", "c"])      # "a,b,c"
```

Dicts Module

```
import dicts

let config = {"host": "localhost", "port": 8080, "debug": false}

print dicts.size(config)           # 3
print dicts.get_or(config, "timeout", 30) # 30 (default)
print dicts.has_all(config, ["host", "port"]) # true

let entries = dicts.entries(config)
for e in entries:
  print str(e[0]) + " = " + str(e[1])
end
```

Iterator Module

```
import iter

# Infinite counter
for n in iter.count(0, 2):
  if n > 10:
    break
  end
  print n    # 0, 2, 4, 6, 8, 10
```

```

end

# Enumeration
let fruits = ["apple", "banana", "cherry"]
for pair in iter.enumerate_array(fruits):
    print str(pair[0]) + ": " + pair[1]
end

# Take first N
let first5 = iter.take(iter.count(0, 1), 5)

```

Stats Module

```

import stats

let scores = [85, 92, 78, 95, 88, 91, 76]

print stats.mean(scores)           # ~86.4
print stats.min_value(scores)      # 76
print stats.max_value(scores)      # 95
print stats.range_span(scores)     # 19
print stats.sum(scores)            # 605

```

Assert Module

```

import assert

let result = 2 + 2
assert.assert_equal(result, 4, "Basic addition")
assert.assert_true(result > 0, "Positive result")
assert.assert_not_nil(result, "Not nil")
assert.assert_close(3.14159, 3.14, 0.01, "Pi approximation")

```

\newpage

JSON Module

The `json` module is a full JSON parser and serializer written in pure Sage, ported from `cJSON`:

```

import json

# Parse a JSON string
let text = "{\"name\": \"Alice\", \"age\": 30}"
let data = json.cJSON_Parse(text)

# Access values
let name_item = json.cJSON_GetObjectItem(data, "name")
print json.cJSON_GetStringValue(name_item)    # Alice

let age_item = json.cJSON_GetObjectItem(data, "age")
print json.cJSON_GetNumberValue(age_item)     # 30

# Create JSON
let obj = json.cJSON_CreateObject()
json.cJSON_AddStringToObject(obj, "greeting", "hello")
json.cJSON_AddNumberToObject(obj, "count", 42)
json.cJSON_AddBoolToObject(obj, "active", true)

# Serialize to string
let output = json.cJSON_Print(obj)
print output

# Clean up
json.cJSON_Delete(data)

```

```
json.cJSON_Delete(obj)
```

The JSON module passes 88 tests and supports all JSON types: objects, arrays, strings, numbers, booleans, and null.

\newpage

Safety Library

The safety library provides explicit Option type operations for working with potentially-nil values:

```
# Option type constructors
let some_val = Some(42)
let no_val = None()

# Unwrap (panics if None)
let x = unwrap(some_val)    # 42

# Safe access with map
let doubled = map(some_val, proc(v): return v * 2 end)
# doubled is Some(84)

# Check before use
if some_val != nil:
    print unwrap(some_val)
end
```

When `--strict-safety` is enabled, the compiler enforces that all Option-typed values are checked before use, preventing nil-related runtime errors.

\newpage

Part V: Compilation and Backends

\newpage

Compilation Backends

Sage provides three compilation backends, each targeting different use cases.

C Backend

The C backend translates Sage to portable C code:

```
# Emit C source code
./sage --emit-c program.sage -o output.c

# Compile to binary via C (uses gcc/cc)
./sage --compile program.sage -o program

# Run the compiled binary
./program
```

The C backend is the most mature and supports the full language. The generated C code is portable and can be compiled with any C11-compliant compiler.

LLVM Backend

The LLVM backend generates LLVM IR text, then uses `clang` to compile:

```
# Emit LLVM IR
./sage --emit-llvm program.sage -o output.ll

# Compile to binary via LLVM
./sage --compile-llvm program.sage -o program
```

The LLVM backend supports classes, methods, exception handling, bitwise operators, and GPU operations. It links against `llvm_runtime.c` which provides 40+ runtime functions.

Native Assembly Backend

The native backend generates assembly directly for three architectures:

```
# Emit x86-64 assembly
./sage --emit-asm program.sage -o output.s --target x86-64

# Emit aarch64 assembly
./sage --emit-asm program.sage -o output.s --target aarch64

# Emit RISC-V 64-bit assembly
./sage --emit-asm program.sage -o output.s --target rv64

# Compile to native binary
./sage --compile-native program.sage -o program --target x86-64
```

Optimization Levels

All backends support optimization passes:

```
./sage -O0 program.sage # No optimization
./sage -O1 program.sage # Basic: constant folding
./sage -O2 program.sage # Standard: + dead code elimination
./sage -O3 program.sage # Aggressive: + function inlining
```

The optimization passes operate on the AST before code generation:

Level	Passes
-O0	None
-O1	Constant folding
-O2	Constant folding, dead code elimination
-O3	Constant folding, dead code elimination, inlining

Debug Info

```
./sage -g program.sage # Include debug information
```

\newpage

Bare-Metal and OS Development

Sage includes a complete pipeline for building bare-metal kernels that boot and run on QEMU for x86_64, aarch64, and RISC-V 64. The boot libraries generate assembly, linker scripts, and QEMU commands from pure Sage code.

Supported Architectures

Architecture	Boot Method	Serial UART	QEMU Machine
x86_64	Multiboot1 (ELF32)	COM1 port I/O (0x3F8)	default PC
aarch64	Direct ELF load	PL011 MMIO (0x09000000)	virt + cortex-a57
riscv64	Direct ELF load	NS16550 MMIO (0x10000000)	virt (-bios none)

Advanced Boot Infrastructure

SageLang v4.1.5 expands the `os.boot` library with 20+ new modules for building sophisticated multi-stage bootloaders.

Firmware Interaction

The `os.boot.bios` module provides code generation for legacy BIOS interrupts from real mode. This allows Sage-based bootloaders to set video modes, read from disks via LBA extensions, and query system information before switching to protected mode.

```
import os.boot.bios as bios

# Set 80x25 text mode
let asm = bios.set_video_mode(0x03)

# Read disk sectors
let dap = bios.disk_address_packet(lba=2048, count=16, dest=0x20000)
asm = asm + bios.int13_read(drive=0x80, "dap_label")
```

Memory Detection

`os.boot.e820` handles the collection and normalization of the system memory map from BIOS. It can find usable RAM regions and determine the highest physical address available for kernel placement.

```
import os.boot.e820 as e820

# Collect memory map entries
let asm = e820.collect_asm("mmap_buffer")

# Filter for usable RAM regions
let usable = e820.filter_usable(mmap)
```

Mode Transitions

The transition from 16-bit real mode to 64-bit long mode is handled by a sequence of composable modules:

- `os.boot.real_mode`: Real-mode stack setup and far jumps.
- `os.boot.prot_mode`: Entering 32-bit protected mode with a flat GDT.
- `os.boot.long_mode`: Transitioning to 64-bit mode via PAE and PML4 setup.

Kernel Loading and Handoff

- `os.boot.elf_load`: Parses ELF binaries and copies PT_LOAD segments to their physical load addresses.
- `os.boot.handoff`: Defines the SageOS Boot Protocol, allowing the bootloader to pass a structured information block (framebuffer, memory map, ACPI tables) to the kernel entry point.

Architecture-Specific Support

- `os.boot.sbi`: RISC-V Supervisor Binary Interface (SBI) wrappers for S-mode bootloaders.
- `os.boot.psci`: ARM Power State Coordination Interface for secondary core bring-up.

Quick Start: Build a Kernel with Sage

The `os.boot.build` module generates all files needed for a bootable kernel. Here is a complete example that builds and runs on all three architectures:

```
# build_kernel.sage – Generate a bootable kernel for any architecture
gc_disable()
import io
import os.boot.start as start
import os.boot.build as build

# Pick your architecture: "x86_64", "aarch64", or "riscv64"
let arch = "x86_64"

# Generate boot assembly with serial output
let boot_asm = start.generate_boot_asm_mb1("Hello from SageOS!")

# Generate linker script
let linker = build.generate_linker_x86_mb1()

# Write files
io.writefile("boot.S", boot_asm)
io.writefile("linker.ld", linker)

# Print build commands
print "Build:"
print "  as --32 -o boot.o boot.S"
print "  ld -m elf_i386 -T linker.ld -o kernel.elf boot.o"
print "Run:"
print "  " + build.qemu_command("x86_64", "kernel.elf")
```

Example: x86_64 Kernel

Generate boot assembly, compile, and run:

```
gc_disable()
import io
import os.boot.start as start
import os.boot.build as build

# Generate self-contained 32-bit multiboot1 kernel with serial
let asm = start.generate_boot_asm_mb1("Hello from SageOS on x86_64!")
let ld = build.generate_linker_x86_mb1()

io.writefile("boot.S", asm)
io.writefile("linker.ld", ld)
```

Build and run:

```
# Assemble (32-bit ELF for multiboot1 compatibility)
as --32 -o boot.o boot.S

# Link
ld -m elf_i386 -T linker.ld -o kernel.elf boot.o

# Run in QEMU
qemu-system-x86_64 -m 128M -display none -serial mon:stdio -kernel kernel.elf
# Output: Hello from SageOS on x86_64!
```

The generated boot stub initializes COM1 at 115200 baud (8N1), prints the message over serial, and halts with `cli; hlt`.

Example: aarch64 Kernel

```

gc_disable()
import io
import os.boot.start as start
import os.boot.build as build

# Generate aarch64 boot assembly + serial (PL011 at 0x09000000)
let asm = start.emit_start_aarch64("kmain", "stack_top")
asm = asm + build.generate_serial_boot_aarch64()

# Generate C kernel
let kernel_c = build.generate_kernel_c("aarch64", "Hello from SageOS on aarch64!")

io.writefile("boot.S", asm)
io.writefile("kernel.c", kernel_c)

```

Build and run:

```

# Assemble
aarch64-linux-gnu-as -o boot.o boot.S

# Compile kernel C
aarch64-linux-gnu-gcc -ffreestanding -nostdlib -c -o kernel.o kernel.c

# Link (base address 0x40000000 for QEMU virt)
aarch64-linux-gnu-ld -T linker.ld -o kernel.elf boot.o kernel.o

# Run in QEMU
qemu-system-aarch64 -machine virt -cpu cortex-a57 -m 128M \
    -display none -serial mon:stdio -kernel kernel.elf
# Output: Hello from SageOS on aarch64!

```

The boot stub disables interrupts (DAIF), sets up the stack, zeroes BSS, then jumps to kmain. The serial driver uses the PL011 UART with FIFO enabled.

Example: RISC-V 64 Kernel

```

gc_disable()
import io
import os.boot.start as start
import os.boot.build as build

# Generate riscv64 boot assembly + serial (NS16550 at 0x10000000)
let asm = start.emit_start_riscv64("kmain", "stack_top")
asm = asm + build.generate_serial_boot_riscv64()

# Generate C kernel
let kernel_c = build.generate_kernel_c("riscv64", "Hello from SageOS on riscv64!")

io.writefile("boot.S", asm)
io.writefile("kernel.c", kernel_c)

```

Build and run:

```

# Assemble
riscv64-linux-gnu-as -march=rv64gc -mabi=lp64d -o boot.o boot.S

# Compile kernel C
riscv64-linux-gnu-gcc -ffreestanding -nostdlib -march=rv64gc \
    -mabi=lp64d -c -o kernel.o kernel.c

# Link (base address 0x80000000 for QEMU virt)
riscv64-linux-gnu-ld -T linker.ld -o kernel.elf boot.o kernel.o

# Run in QEMU (must use -bios none to skip OpenSBI)
qemu-system-riscv64 -machine virt -m 128M \
    -display none -serial mon:stdio -bios none -kernel kernel.elf

```

```
# Output: Hello from SageOS on riscv64!
```

The boot stub disables machine-mode interrupts (mstatus.MIE), sets up the stack, zeroes BSS, and calls `kmain`. The serial driver uses a 16550-compatible UART in MMIO mode.

Boot Library Reference

Module	Purpose
<code>os.boot.start</code>	Boot assembly generation (multiboot, long mode, BSS init)
<code>os.boot.build</code>	Build pipeline (serial drivers, kernel templates, QEMU commands)
<code>os.boot.linker</code>	Linker script generation (x86_64, aarch64, riscv64)
<code>os.boot.multiboot</code>	Multiboot2 header construction and parsing
<code>os.boot.gdt</code>	GDT descriptor tables (x86_64 long mode)
<code>os.serial</code>	UART configuration and assembly emission (3 architectures)
<code>os.qemu</code>	QEMU VM configuration and command generation

Kernel Libraries

Module	Purpose
<code>os.kernel.kmain</code>	Kernel initialization pipeline (6 phases)
<code>os.kernel.console</code>	VGA text mode + framebuffer console
<code>os.kernel.pmm</code>	Physical memory manager (bitmap allocator)
<code>os.kernel.vmm</code>	Virtual memory manager (4-level paging)
<code>os.idt</code>	Interrupt descriptor tables (x86/aarch64/riscv64)
<code>os.uefi</code>	UEFI memory map and ACPI table parsing
<code>os.acpi</code>	MADT, FADT, HPET, MCFG parsers

Debugging with GDB

All architectures support GDB debugging via QEMU's `-s -S` flags:

```
# Start QEMU paused, waiting for GDB on port 1234
qemu-system-x86_64 -m 128M -display none -serial mon:stdio \
    -kernel kernel.elf -s -S &

# Connect GDB
gdb kernel.elf -ex 'target remote :1234' -ex 'break _start' -ex 'continue'
```

For cross-architecture debugging, use the appropriate GDB:

```
# aarch64
aarch64-linux-gnu-gdb kernel.elf -ex 'target remote :1234'

# riscv64
riscv64-linux-gnu-gdb kernel.elf -ex 'target remote :1234'
```

\newpage

Part VI: Tooling

\newpage

Developer Tools

REPL

Launch the interactive read-eval-print loop:

```
./sage
# or
./sage --repl

sage> 2 + 3
5
sage> let x = "hello"
sage> print upper(x)
HELLO
sage> proc square(n): return n * n end
sage> print square(7)
49
sage> :quit
```

The REPL supports multiline input, error recovery, and all language features. Commands like `:stats` can be used to view interpreter and GC statistics.

Formatter

Automatically format Sage source files:

```
./sage --fmt program.sage
# or
./sage fmt program.sage
```

The formatter normalizes indentation, spacing, and code layout. Both C and self-hosted (Sage) implementations are available.

Lint

Check code for style issues and common mistakes:

```
./sage --lint program.sage
# or
./sage lint program.sage
```

The linter reports unused variables, naming convention violations, overly complex functions, and other code quality issues.

Type Checker

Run static type analysis:

```
./sage check program.sage
```

The type checker performs best-effort type inference and reports type mismatches, unreachable code, and annotation violations.

Safety Analyzer

Run the compile-time safety analysis:

```
./sage safety program.sage
./sage --strict-safety program.sage
```

Reports ownership violations, borrow conflicts, lifetime errors, and nil-safety issues.

Language Server (LSP)

Start the LSP server for editor integration:

```
./sage --lsp
# or
./sage-lsp
```

The LSP server provides:

- **Autocompletion** of functions, variables, and module members
- **Diagnostics** (errors and warnings as you type)
- **Hover info** (type information and documentation)
- **Go-to-definition** navigation

Both C and self-hosted implementations exist. Compatible with VS Code, Neovim, and any editor supporting the Language Server Protocol.

\newpage

Build System

Makefile Targets

```
make                # Build the sage interpreter
make test           # Run interpreter tests (269+ tests)
make test-selfhost  # Run self-hosted tests (1567+ tests)
make test-all      # Run all tests

# Bare-metal / QEMU
make qemu-bare      # Run kernel in QEMU (x86_64)
make qemu-bare-arm64 # Run kernel in QEMU (aarch64)
make qemu-debug     # Debug kernel with GDB
make kernel-bare    # Compile bare-metal kernel
make kernel-uefi    # Compile UEFI application
```

Version

The version is stored in a single VERSION file at the repository root. All build systems (Makefile, CMakeLists.txt, sagemake) read from this file automatically. Current version: 3.

\newpage

Part VIb: Metaprogramming

\newpage

Compile-Time Execution

Sage supports compile-time code execution through the `comptime` keyword. Because Sage has a built-in interpreter, any Sage code can run during compilation to produce constants, lookup tables, or configuration data that is baked directly into

the binary.

Comptime Blocks

A `comptime` block executes its body at compile time. Variables defined inside a `comptime` block are available to subsequent code.

```
# Pre-compute a lookup table
comptime:
  let SINE_TABLE = []
  for i in range(256):
    push(SINE_TABLE, math.sin(i * math.pi / 128.0))
  end
end

proc get_sine(angle):
  return SINE_TABLE[angle % 256]
end
```

Comptime Expressions

The `comptime(expr)` form evaluates a single expression at compile time:

```
proc factorial(n):
  if n <= 1:
    return 1
  return n * factorial(n - 1)

# Computed at compile time, baked as a constant
let FACT_10 = comptime(factorial(10))
```

\newpage

Pragmas and Decorators

Pragmas attach metadata to declarations using the `@` symbol. They inform the compiler backends how to handle specific functions, structs, or blocks.

Syntax

```
@pragma_name
@pragma_name("argument")
```

Built-in Pragmas

Pragma	Target	Effect
@inline	proc	Hints the C backend to emit <code>static inline</code>
@packed	struct	Emits <code>#pragma pack(push, 1)</code> for no padding
@section("name")	proc	Places function in a specific ELF section
@align("N")	struct	Alignment hint for struct layout
@deprecated	proc	Marks a function as deprecated
@noreturn	proc	Marks a function that never returns

Examples

```

@packed
struct Ipv4Header:
  version_ihl: u8
  tos: u8
  total_length: u16

@inline
proc fast_math(x, y):
  return x * y

@section(".multiboot_header")
proc boot_header():
  pass

```

Multiple Pragmas

Multiple pragmas can be stacked on a single declaration:

```

@inline
@deprecated
proc old_multiply(a, b):
  return a * b

```

\newpage

AST Macros

Macros are functions that execute at compile time and operate on code structure. In the current implementation, macros are defined with the `macro` keyword and behave as compile-time procedures.

Macro Definition

```

macro log_call(label):
  print "entering " + label
  # ... macro body ...
  print "exiting " + label

log_call("main")

```

In interpreter mode, macros are treated as regular functions. In compiled mode, macro bodies are expanded at the call site during compilation.

Future: Quote and Unquote

The `quote` and `unquote` keywords are reserved for future AST macro support, enabling macros that manipulate Abstract Syntax Tree nodes directly:

```

macro safe_zone(body):
  return quote:
    sys.enable_strict_safety()
    defer sys.restore_safety()
    unquote(body)
  end

```

\newpage

Generics

Sage supports generic type parameters on procedures and structs using bracket syntax `[T, U, ...]`. In the current implementation, Sage uses dynamic typing, so generic parameters serve as documentation and enable future monomorphization in compiled backends.

Generic Procedures

```
proc identity[T](x: T) -> T:
  return x

proc swap[T](a: T, b: T):
  let tmp = a
  a = b
  b = tmp
```

Generic Structs

```
struct Pair[A, B]:
  first: A
  second: B

struct Stack[T]:
  items: Array[T]
  size: Int
```

Monomorphization (Planned)

When compiled with the C or LLVM backend, the compiler will generate specialized versions for each concrete type used:

```
# At compile time, generates Stack_Int and Stack_String
let int_stack = Stack[Int]()
let str_stack = Stack[String]()
```

\newpage

Part VIc: Garbage Collection

GC Modes

Sage provides three garbage collection strategies, selectable at startup:

```
sage --gc:tracing file.sage # Default: concurrent mark-sweep
sage --gc:arc file.sage    # Automatic Reference Counting
sage --gc:orc file.sage    # Optimized RC with cycle detection
```

Tracing GC (Default)

Concurrent tri-color mark-sweep with SATB write barriers:

- Sub-millisecond stop-the-world pauses (root scan ~50-200us)
- Adaptive threshold triggering based on allocation pressure
- New objects born BLACK (allocated-black invariant)
- Sweeping in 256-object batches to bound pause times

ARC (Automatic Reference Counting)

Deterministic memory reclamation via reference counts:

- Objects freed immediately when reference count reaches zero
- Predictable memory usage (no deferred collection)
- Convenience macros: `ARC_RETAIN`, `ARC_RELEASE`, `ARC_ASSIGN`
- Cannot collect reference cycles — use ORC for cyclic structures

ORC (Optimized Reference Counting)

Nim-inspired ORC combining ARC with Lins' trial deletion cycle collector:

- Three-phase cycle detection: mark PURPLE candidates, trial-decrement scan, collect WHITE garbage
- Triggers cycle collection every 500 decrements (more aggressive than ARC's 1000)
- Recommended for programs with complex object graphs (linked lists, trees, circular references)

GC API

```
gc_collect()      # Force a collection cycle
gc_stats()       # Print GC statistics
gc_enable()      # Enable automatic collection
gc_disable()     # Disable automatic collection (manual only)
gc_set_arc()     # Switch to ARC mode at runtime
gc_set_orc()     # Switch to ORC mode at runtime
gc_mode()       # Returns "tracing", "arc", or "orc"
```

\newpage

Part VIId: Kotlin/Android Backend

Sage can transpile to Kotlin and generate complete Android projects from a single `.sage` file.

Emitting Kotlin

```
sage --emit-kotlin app.sage -o app.kt      # Transpile to Kotlin
sage --emit-kotlin app.sage -o app.kt -O2 # With type specialization
```

The transpiler maps all Sage constructs to idiomatic Kotlin:

Sage	Kotlin
<code>let x = 10</code>	<code>var x = S.num(10.0)</code>
<code>proc foo(a):</code>	<code>fun foo(a: SageVal): SageVal</code>
<code>class Dog(Animal):</code>	<code>open class Dog : Animal()</code>
<code>for x in items:</code>	<code>for (x in S.toIterable(items))</code>
<code>match x:</code>	<code>when-chain with S.equal()</code>
<code>try: ... catch e:</code>	<code>try { } catch (_e: SageException)</code>
<code>yield val</code>	<code>yield(val) inside sequence { }</code>
<code>await expr</code>	<code>kotlinx.coroutines.runBlocking { expr }</code>
<code>super.init(x)</code>	<code>super.sageInit(x)</code>

Building Android Apps

```
sage --compile-android app.sage -o my_app \
    --package com.example.app \
    --app-name "My App" \
    --min-sdk 24
```

This generates a complete Gradle project:

```
my_app/
  build.gradle.kts
  settings.gradle.kts
  app/
    build.gradle.kts
    src/main/
      AndroidManifest.xml
      kotlin/<package>/Main.kt          # Transpiled Sage
      kotlin/<package>/MainActivity.kt   # Android launcher
      kotlin/sage/runtime/SageRuntime.kt
      res/values/strings.xml, styles.xml
```

Build with: `cd my_app && ./gradlew assembleDebug`

Type Specialization (-O2+)

At optimization level 2+, the transpiler emits native Kotlin types for variables initialized with literals:

```
let x = 10          # emits: var x: Double = 10.0
let name = "Sage"   # emits: var name: String = "Sage"
let flag = true     # emits: var flag: Boolean = true
```

This eliminates SageVal boxing overhead on hot paths.

Generators

Generator functions transpile to Kotlin `sequence { }` blocks:

```
proc count_up(n):
  let i = 0
  while i < n:
    yield i
    i = i + 1
```

Emits:

```
fun count_up(n: SageVal): Sequence<SageVal> = sequence {
  var i = S.num(0.0)
  while (S.truthy(S.lt(i, n))) {
    yield(i)
    i = S.add(i, S.num(1.0))
  }
}
```

Async/Await (Coroutines)

Async procs emit `suspend fun`; `await` uses `kotlinx.coroutines.runBlocking`:

```
async proc fetch():
  return 42

let result = await fetch()
```

Memory Operations on Android

`mem_alloc/mem_read/mem_write/mem_free` map to `java.nio.ByteBuffer`:

```
let buf = mem_alloc(256)
mem_write(buf, 0, "int", 42)
print(mem_read(buf, 0, "int")) # 42
mem_free(buf)
```

Jetpack Compose

When `import android.compose` is detected, the project generator uses Compose:

- `@Composable` Activity with Material 3
- Compose BOM, navigation-compose, ui-tooling dependencies
- `ComponentActivity + setContent { }` instead of programmatic views

GPU Graphics on Android

The `lib/android/graphics.sage` module provides Vulkan-style and OpenGL ES APIs:

```
import android.graphics

let gpu = GPUContext("My App")
gpu.initialize()

# Create compute buffer and run shader
let buf = gpu.create_buffer(1024, "storage")
gpu.upload(buf, [1.0, 2.0, 3.0])
gpu.dispatch_compute("shader.comp", buf, 256)
let result = gpu.download(buf)

# OpenGL ES convenience
let gl = GLESContext()
gl.initialize()
gl.clear(0.1, 0.1, 0.2, 1.0)
gl.draw_arrays("triangles", 0, 3)
gl.swap_buffers()
```

Concurrency on Android

Kotlin transpiler maps Sage concurrency to JVM primitives:

```
# Atomics → java.util.concurrent.atomic.AtomicLong
let counter = atomic_new(0)
atomic_add(counter, 1)

# Semaphores → java.util.concurrent.Semaphore
let sem = sem_new(3)
sem_wait(sem)
sem_post(sem)

# CPU info (via Runtime.getRuntime())
let cores = cpu_count()
```

Additional Mapped Builtins

The Kotlin transpiler now maps 60+ built-in functions:

- **Strings:** `upper`, `lower`, `strip`, `split`, `join`, `replace`, `chr`, `ord`
- **Paths:** `path_join`, `path_exists`, `path_basename`, `path_dirname`, `path_ext`
- **Timing:** `clock()` (`System.nanoTime`)

- **Hashing:** `hash(v)` (`Object.hashCode`)

\newpage

Part VIe: Performance Optimization

Hybrid JIT/AOT Architecture

The self-hosted interpreter implements a profile-guided type specialization engine that runs automatically (no flags needed):

How It Works

1. Profiling Phase: Every function call records the argument types and increments a call counter. After `HOT_THRESHOLD` (50) calls, the function is analyzed.

2. Monomorphic Detection: If all observed calls passed the same argument types (e.g., always numbers), the function is marked as "specialized".

3. Type-Feedback Interpretation: Specialized functions always hit the number fast-path in `eval_binary` (inline arithmetic, no dispatch table lookup). This gives 100% fast-path rate vs ~70% without profiling.

4. Loop Specialization: While-loops profile the first 8 iterations. If the body never returns break/return/continue signals, the loop switches to a "fast mode" that skips signal checking entirely — a ~30% speedup on tight loops.

Example

```
proc fib(n):
  if n <= 1:
    return n
  return fib(n - 1) + fib(n - 2)

# After ~50 calls, fib() is profiled as monomorphic (all-number args).
# The number fast-path in eval_binary is taken on every binary operation.
print fib(28)
```

Profiling API

```
# These are internal but accessible:
let profile = _get_profile("fib") # Get profile for a function
# profile["calls"] → 317811 (call count)
# profile["arg_types"] → ["number"]
# profile["monomorphic"] → true
# profile["specialized"] → true
```

Performance Library (`lib/perf.sage`)

The `perf` module provides reusable optimization primitives:

Signal Singletons

Eliminate dict allocation on every statement return:

```
import perf
```

```
# Instead of allocating {"kind": 0, "value": nil} every time:
let result = perf.sig_normal_nil() # Returns pre-allocated singleton
let brk = perf.sig_break()        # Zero allocation
```

Dispatch Tables

Replace if/elif chains with O(1) dict lookup:

```
import perf

let table = perf.make_dispatch_table()
perf.dispatch_register(table, "add", proc(args): return args[0] + args[1])
perf.dispatch_register(table, "sub", proc(args): return args[0] - args[1])

# O(1) dispatch instead of N comparisons:
let result = perf.dispatch_call(table, "add", [10, 20])
```

Shape Objects

Pre-shaped dict constructors for known structures:

```
import perf

let func = perf.shape_function("add", params, body, closure, false)
let cls = perf.shape_class("Dog", parent)
let inst = perf.shape_instance(cls)
let env = perf.shape_env(parent_env)
```

Fast Numeric Operations

Bypass type dispatch for known-numeric paths:

```
import perf

let sum = perf.fast_sum([1, 2, 3, 4, 5])
let total = perf.fast_add_num(a, b)
```

Flat Environment Cache

O(1) variable access bypassing scope chains in tight loops:

```
import perf

let cache = perf.flat_cache_snapshot(env)
# ... hot loop using flat_cache_get/set ...
perf.flat_cache_flush(cache, env)
```

Self-Hosted Interpreter Optimizations

The self-hosted interpreter (`src/sage/interpreter.sage`) applies these patterns:

- 1. Pre-allocated signal singletons:** `result_normal(nil)`, `result_break()`, `result_continue()` return cached dicts
- 2. Native dispatch table:** `call_native()` uses O(1) dict lookup instead of 180-line if/elif chain
- 3. Binary op dispatch table:** `eval_binary()` uses O(1) dict lookup for all operators
- 4. Recursion depth at call boundaries only:** `eval_expr()` no longer increments/decrements depth counter
- 5. Shape constructors:** functions, classes, environments built as single dict literals

Native C Interpreter Optimizations

The C interpreter (`src/c/interpreter.c`, `src/c/env.c`) applies:

1. **Cached name length in EnvNode:** `name_length` field avoids `strlen` during lookup
2. **memcmp with length pre-check:** `env_get/env_define/env_assign` check `name_length == length` before `memcmp`
3. **Inlined eval_expr:** a stack-proximity guard (`stack_danger()`) checks real stack usage at both `interpret()` and per-expression boundaries, so deep expression nesting fails with a catchable exception before the OS stack is exhausted (v4.1.16)
4. **For-loop slot caching:** loop variable node pointer cached after first `env_define`, subsequent iterations write directly
5. **String pointer equality:** `values_equal()` checks `AS_STRING(a) == AS_STRING(b)` before `strcmp`

Algorithmic Optimizations (v4.1.5)

Recent updates have transitioned key library operations from interpreted loops to native C implementations or more efficient algorithms:

1. **Dictionary Size:** `dicts.size(d)` now uses the native `len(d)` builtin, achieving $O(1)$ complexity (~590x speedup).
2. **Array Uniqueness:** `arrays.unique(arr)` now uses a dictionary for lookups, achieving $O(N)$ average-case complexity (previously $O(N^2)$).
3. **Array Reversal:** `arrays.reverse(arr)` now uses a native C implementation (`array_reverse`), achieving ~105x speedup.
4. **String/Value Repeating:** `strings.repeat` and `utils.repeat_value` now use binary exponentiation and `array_extend`, achieving $O(\log n)$ efficiency (~13x speedup).
5. **Array Partitioning:** `arrays.take` and `arrays.drop` now use native `slice`, achieving ~88x speedup.

Cross-Backend Benchmarks

Run the 8-workload benchmark across all backends:

```
bash benchmarks/run_backend_compare.sh
python3 scripts/generate_backend_chart.py
```

Workloads: fibonacci, loop sum, array ops, string concat, dict ops, prime sieve, nested loops, LCG hash.

\newpage

Part VI: Default Runtime and Execution Modes

JIT+AOT Hybrid Default

As of v4.1.5, Sage's default runtime is `auto` — JIT profiling mode on hosted platforms, AST interpreter on bare-metal:

Environment	Auto Resolves To	Why
Desktop/Server	JIT profiling	Full OS, type feedback collection
Android	JIT profiling	JVM + coroutines available
Bare-metal/Pico	AST interpreter	No fork/system, safe fallback
Kernel dev	SageMetal VM	Freestanding, zero allocation

Override with `--runtime ast`, `--runtime bytecode`, `--runtime jit`, or `--runtime aot`.

Runtime Pragma Decorators

Control JIT/AOT behavior per-function:

```
@nojit
proc low_level_driver():
    # Skips JIT profiling for this function
    mem_write(ptr, 0, "int", 42)

@noaot
proc dynamic_dispatch():
    # Skips AOT compilation
    ...

@noprofile
proc realtime_handler():
    # Skips all profiling overhead
    ...
```

\newpage

Part VIg: SageMetal VM — Bare-Metal Bytecode Interpreter

The SageMetal VM is a freestanding bytecode interpreter that runs without an OS, libc, or dynamic memory allocation. It is designed for kernels, bootloaders, embedded systems, and OS development.

Architecture

All storage is static — no malloc, no heap fragmentation:

- **Value Stack:** 512 slots (configurable via `METAL_STACK_SIZE`)
- **Constant Pool:** 1024 entries
- **String Pool:** 32KB bump allocator (interned, deduplicated)
- **Heap:** 64KB bump allocator for general use
- **Scope Chain:** 64 levels deep, 32 variables per scope
- **Array Pool:** Fixed-capacity arrays (256 elements each)

MetalValue Type

Compact 16-byte tagged union:

```
MV_NIL    — null value
MV_NUM    — IEEE 754 double
MV_BOOL   — 0 or 1
MV_STR    — index into string pool
```

```

MV_ARR    - index into array pool
MV_DICT   - index into dict pool
MV_FN     - index into function table
MV_PTR    - raw pointer (for MMIO, DMA)

```

Host I/O Callbacks

The kernel or bootloader sets these before running the VM:

```

MetalVM vm;
metal_vm_init(&vm);
vm.write_char = serial_putchar; // Console output
vm.read_char = serial_getchar;  // Console input
vm.write_port = x86_outb;       // Port I/O
vm.read_port = x86_inb;         // Port I/O
vm.map_mmio = identity_map;     // MMIO mapping

metal_vm_load(&vm, bytecode, length);
metal_vm_run(&vm); // Execute until halt

```

Single-Step Mode

For cooperative multitasking in kernels:

```

while (metal_vm_step(&vm)) {
    // Check interrupts, handle timer ticks, etc.
    if (timer_interrupt_pending) handle_timer();
}

```

Building

```
make metal-vm # Produces obj/metal_vm.o (freestanding)
```

Link with your kernel:

```
gcc -ffreestanding -nostdlib -o kernel.elf \
    boot.o kernel.o obj/metal_vm.o -lgcc
```

\newpage

Part VIh: Metal Standard Library

The `lib/metal/` modules provide bare-metal primitives for kernel and embedded development.

metal.core

Core primitives: serial I/O, port I/O, MMIO, CPU control, memory.

```

import metal.core

# Console
core.puts("Hello from kernel!")

# Port I/O (x86)
core.outb(0x3F8, 65) # Send 'A' to COM1
let val = core.inb(0x60) # Read keyboard scancode

# MMIO
core.mmio_write32(0xB8000, 0x0F41) # Write 'A' to VGA

# CPU control & Memory barriers
core.cli() # Disable interrupts

```



```

core.sti()    # Enable interrupts
core.hlt()    # Halt until next interrupt
core.dmb()    # ARM Data Memory Barrier
core.dsb()    # ARM Data Synchronization Barrier
core.isb()    # ARM Instruction Synchronization Barrier
core.fence()  # RISC-V Memory Fence

# Multicore & Synchronization
let cid = core.cpu_id()
core.critical_section_enter()
# ... atomic operation ...
core.critical_section_exit()

core.spin_lock(0x1000)
# ... protected section ...
core.spin_unlock(0x1000)

# Bump allocator
core.heap_init(0x100000, 65536)
let ptr = core.heap_alloc(256)

```

metal.serial

UART drivers for x86 (NS16550A) and ARM (PL011):

```

import metal.serial

# x86 COM1
serial.uart_init(serial.COM1, 115200)
serial.uart_puts(serial.COM1, "Boot complete\n")

# ARM PL011
serial.pl011_init(0x09000000)
serial.pl011_puts(0x09000000, "Hello from ARM\n")

# Utility
if serial.baud_rate_valid(115200):
    serial.uart_init(serial.COM1, 115200)
end

# Advanced I/O
let line = serial.uart_readline(serial.COM1)
let byte = serial.uart_read_timeout(serial.COM1, 1000) # 1s timeout
serial.uart_flush_rx(serial.COM1)

```

Serial API

Function	Description
uart_init(port, baud)	Initialize NS16550A COM port at specified baud rate.
uart_init_ext(port, baud, bits, stop, parity)	Extended COM port initialization.
uart_send(port, byte)	Transmit single byte (blocking).
uart_recv(port)	Receive single byte (blocking).
uart_puts(port, s)	Transmit string over COM port.
uart_read_timeout(port, timeout_ms)	Receive single byte with timeout in milliseconds (returns nil on timeout).
uart_readline(port)	Read line (until newline/CR) with terminal echo and backspace support.
uart_flush_rx(port)	Flush pending bytes in RX buffer.
pl011_init(base)	Initialize ARM PL011 UART at base address.
pl011_send(base, byte)	Transmit single byte via PL011.

Function	Description
<code>pl011_recv(base)</code>	Receive single byte via PL011 (blocking).
<code>pl011_puts(base, s)</code>	Transmit string via PL011.
<code>pl011_read_timeout(base, timeout_ms)</code>	Receive single byte via PL011 with timeout.
<code>pl011_readline(base)</code>	Read line via PL011 with terminal echo and backspace support.
<code>pl011_flush_rx(base)</code>	Flush pending bytes in PL011 RX buffer.

metal.irq

Interrupt management and PIC control:

```
import metal.irq

# Remap PIC to vectors 32-47
irq.pic_remap(32, 40)

# Register handler for timer interrupt
irq.register_handler(32, proc(vector):
    irq.irq_enter() # Track nesting depth
    # Handle timer tick
    irq.pic_eoi(irq.IRQ_TIMER)
    irq.irq_exit()
)

# Enable timer IRQ
irq.unmask_irq(irq.IRQ_TIMER) # Arch-neutral helper
```

IRQ API

Function	Description
<code>register_handler(v, f)</code>	Register function <code>f</code> for vector <code>v</code> . Panics if already registered.
<code>register_handler_safe(v, f)</code>	As <code>register_handler</code> , but returns <code>false</code> on a duplicate and keeps the existing handler instead of panicking. Use when handlers are assembled at runtime and a collision is recoverable.
<code>unregister_handler(v)</code>	Remove the handler for <code>v</code> , returning <code>true</code> if there was one. Returns <code>false</code> rather than panicking when there is none, so unregistration is idempotent.
<code>set_priority(v, l)</code>	Set interrupt priority level <code>l</code> for vector <code>v</code> .
<code>get_priority(v)</code>	Get current priority level for vector <code>v</code> .
<code>irq_enter()</code>	Increment nesting depth.
<code>irq_exit()</code>	Decrement nesting depth.
<code>irq_depth()</code>	Returns current nesting depth.
<code>mask_irq(n)</code>	Mask (disable) IRQ line <code>n</code> .
<code>unmask_irq(n)</code>	Unmask (enable) IRQ line <code>n</code> .

metal.timer

Hardware timer with tick counting and sleep:

```
import metal.timer

timer.timer_init_periodic(1000) # 1000 Hz (1ms resolution)

let start = timer.ticks()
# ... do work ...
let elapsed = timer.stopwatch_elapsed_ms(start)
```

```

timer.sleep_ms(500)      # Sleep 500ms

# Query remaining time in current cycle
let remaining = timer.timer_remaining_ms()

```

Timer API

Function	Description
<code>timer_init_periodic(hz)</code>	Initialize PIT in periodic mode.
<code>timer_init_one-shot(hz)</code>	Initialize PIT in one-shot mode.
<code>timer_remaining_ms()</code>	Get remaining milliseconds in current cycle via hardware latching.
<code>timer_cancel_safe()</code>	Safely cancel timer via IRQ masking.
<code>ticks()</code>	Get total ticks since initialization.

metal.gpio

GPIO pin control for embedded targets:

```

import metal.gpio

gpio.gpio_init(0x40000000, 32) # Init 32 pins at MMIO base
gpio.pin_mode(25, gpio.PIN_OUTPUT) # LED pin
gpio.led_blink(25, 5, 250)      # Blink 5 times, 250ms interval

# Batch operations
gpio.pin_set_mask(0b1011)      # Set pins 0, 1, 3 HIGH
gpio.pin_clear_mask(0b1011)    # Set pins 0, 1, 3 LOW
gpio.pin_write_masked(0b11, 0b10) # Set pin 1 HIGH, pin 0 LOW

```

GPIO API

Constants:

- **Modes:** `PIN_INPUT`, `PIN_OUTPUT`, `PIN_ALT`, `PIN_ANALOG`
- **Levels:** `PIN_LOW`, `PIN_HIGH`
- **Pulls:** `PULL_NONE`, `PULL_UP`, `PULL_DOWN`
- **Interrupts:** `INT_DISABLED`, `INT_RISING`, `INT_FALLING`, `INT_BOTH`, `INT_LOW`, `INT_HIGH`

Function	Description
<code>pin_mode(p, m)</code>	Set mode (INPUT, OUTPUT, ALT, ANALOG).
<code>pin_get_mode(p)</code>	Get current pin mode.
<code>pin_pull(p, cfg)</code>	Set pull configuration (NONE, UP, DOWN).
<code>pin_get_pull(p)</code>	Get current pull configuration.
<code>pin_set_interrupt(p, m)</code>	Set interrupt trigger (RISING, FALLING, BOTH, etc).
<code>pin_get_interrupt(p)</code>	Get current interrupt mode.
<code>pin_register_handler(p, f)</code>	Register callback <code>f</code> for pin <code>p</code> .
<code>pin_enable_interrupt(p)</code>	Enable interrupt for pin <code>p</code> .
<code>pin_disable_interrupt(p)</code>	Disable interrupt for pin <code>p</code> .
<code>gpio_dispatch(p)</code>	Manually dispatch interrupt for pin <code>p</code> .
<code>pin_debounce(p, s, n, d)</code>	Debounce pin <code>p</code> to state <code>S</code> .
<code>led_on(p)</code>	Turn LED on pin <code>p</code> ON.

Function	Description
<code>led_off(p)</code>	Turn LED on pin <code>p</code> OFF.
<code>led_blink(p, n, d)</code>	Blink LED on pin <code>p</code> , <code>n</code> times with delay <code>d</code> .
<code>pin_set_mask(m)</code>	Set multiple pins HIGH.
<code>pin_clear_mask(m)</code>	Set multiple pins LOW.
<code>pin_write_masked(m, v)</code>	Write values to multiple pins.

ESP32 Microcontroller Board Support (core/boards/ESP32, core/lib/esp32.sage)

SageLang provides native board support for the classic **ESP32** (ESP32-D0WD-V3 / DevKitC, 4MB SPI flash): dual-core Xtensa LX6 @ 240MHz with 2.4GHz WiFi.

- **Board Support Module:** `import esp32` provides chip identification constants (`CHIP_NAME`, `CPU_ARCH`, `CPU_CORES`, `CPU_FREQ_MHZ`), flash layout offsets (`FLASH_BOOTLOADER_OFFSET`, `FLASH_PART_TABLE_OFFSET`, `FLASH_APP_OFFSET`), pin capability predicates (`pin_valid`, `pin_can_output`, `pin_is_input_only`, `pin_is_strapping`, `pin_is_flash`), ADC1 voltage conversion (`adc1_channel`, `adc_to_mv`), touch pad mapping (`touch_channel`), deep-sleep wake pin mapping (`rtc_capable`), LEDC PWM duty count calculation (`pwm_duty`), UART remap detection (`uart_needs_remap`), and verified `esptool` flash command generation (`esptool_write_cmd`).
- **Board Package:** `core/boards/ESP32/` includes `__init__.sage`, hardware bring-up examples (`examples/blink.sage`, `examples/hello.sage`), and unit tests (`test_smoke.sage`).

```
import esp32

print esp32.describe()
if esp32.pin_can_output(2):
    let duty = esp32.pwm_duty(75, 8) # 75% duty cycle on 8-bit LEDC timer
    print "PWM Duty: " + str(duty)

let mv = esp32.adc_to_mv(2048, 11) # 2048 raw reading at 11dB attenuation
print "ADC1 Voltage (mV): " + str(mv)
```

Bare-Metal Kernel Example

Combining the Metal library into a minimal "Hello World" kernel with keyboard input:

```
import metal.core as core
import metal.serial as serial
import metal.irq as irq
import metal.timer as timer

proc kernel_main():
    # 1. Initialize hardware
    serial.uart_init(serial.COM1, 115200)
    timer.pit_init(100) # 100Hz
    irq.pic_remap(32, 40)

    serial.uart_puts(serial.COM1, "SageMetal Kernel v1.0 Booting...\n")

    # 2. Setup keyboard interrupt (vector 33 for IRQ 1)
    irq.register_handler(33, proc(vector):
        let scancode = core.inb(0x60)
        serial.uart_puts(serial.COM1, "Key pressed: " + str(scancode) + "\n")
        irq.pic_eoi(1)
    )
    irq.pic_unmask(1) # Keyboard

    # 3. Main Loop
```

```

serial.uart_puts(serial.COM1, "Kernel ready. Type something...\n")
core.sti() # Enable interrupts

var last_print = 0
while true:
    let now = timer.ticks()
    if now - last_print > 500: # Every 5 seconds at 100Hz
        serial.uart_puts(serial.COM1, "[Heartbeat] Uptime: " + str(now/100) + "s\n")
        last_print = now
    end
    core.hlt() # Wait for next interrupt
end
end
\newpage

```

Part VII: Blockchain and Distributed Ledger Technology

The `lib/blockchain/` library provides a pure SageLang implementation of an enterprise-grade L1 blockchain.

Architecture

The system is highly modular, with core components separating concerns:

- **Blockchain Core** (`blockchain.blockchain`): Manages the main ledger state, mempool, and consensus coordination.
- **Ledger Storage** (`blockchain.db`): Provides high-performance, disk-backed storage for blocks, transactions, and world state.
- **Consensus Engines** (`blockchain.consensus.*`): Pluggable architecture supporting:
 - **Proof-of-Work (PoW)**: Standard mining difficulty adjustment.
 - **Proof-of-Authority (PoA)**: Validator-based consensus with automatic slashing for equivocation.
- **World State Trie** (`blockchain.merkle`): A persistent Merkle-Radix Trie for global account balances and contract state, ensuring cryptographically verifiable `state_roots`.
- **Network Layer** (`blockchain.net`, `blockchain.rpc`): Implements P2P node discovery, Initial Block Download (IBD), fork resolution, and a standard JSON-RPC 2.0 API for dApp integration.

Key Features

- **Smart Contracts & NFTs**: Includes a native VM with gas metering and support for inter-contract calls. The **SNFT-721 standard** provides full support for non-fungible tokens.
- **Wallet & Security**: Supports BIP-39 style deterministic HD wallets for address generation and transaction signing. If `libsage_crypto.so` (Ed25519) is available via FFI, it provides high-performance hardware-accelerated signatures.
- **Priority Fee Market**: Mempool implementation that prioritizes transactions based on `gas_price`, dynamically incentivizing miners.
- **Orbit Dynamic Mining**: A dynamic mining rate model that adjusts rewards based on network adoption, total supply, and node reliability.
- **Staking System**: Logic for ORBIT token staking via system smart contracts to provide passive rewards (~5% APR).

Usage Example

The following example demonstrates how to set up a blockchain with PoW, create a wallet, add and sign a transaction, and mine a block:

```
import blockchain.blockchain as bc_mod
import blockchain.wallet as wallet_mod
import blockchain.consensus.pow as pow_mod
import blockchain.transaction as tx_mod

# Initialize a blockchain with PoW consensus
let consensus = pow_mod.PowConsensus(nil, 2)
let coin = bc_mod.Blockchain(consensus, "./sagechain_db")
consensus.blockchain = coin

# Generate a new wallet
let wallet = wallet_mod.Wallet(nil)

# Create and sign a transaction
let tx = coin.add_transaction(wallet.get_address(), "recipient", 100)
wallet.sign_transaction(tx)
coin.add_signed_transaction(tx)

# Mine the pending transactions
coin.mine_pending_transactions("miner-address")

# Retrieve balance
print coin.get_balance(wallet.get_address())
```

For advanced use cases, including P2P networking and validator configuration, refer to the examples in `examples/blockchain_*.sage`.

\newpage

Part VIII: Discord Bot Library

SageLang now includes a library for building Discord bots, designed to mirror the familiarity of Python's `discord` and `discord.ext` libraries.

Getting Started

To create a Discord bot, import the `discord.client` module:

```
import discord.client

proc on_ready(data):
    print("Bot is ready!")
end

# Intent 32767 = all intents
let bot = discord.client.Client("YOUR_TOKEN", 32767)
bot.on("READY", on_ready)
bot.run()
```

REST API Support

The library includes an HTTP client for interacting with Discord's REST API. You can send messages directly:

```
bot.send_message(CHANNEL_ID, "Hello, world!")
```

Features

- **Gateway API:** Full support for real-time event handling via WebSockets.

- **REST API:** Wrapper for sending messages, managing channels, and more.
- **Event System:** Easy-to-use `on()` listener pattern.

\newpage

Part VIIb: Machine Learning and AI

Large Language Models (LLM)

SageLang provides a comprehensive LLM toolkit in `lib/llm/`.

TurboQuant (ICLR 2026)

TurboQuant provides near-optimal vector quantization for KV cache and weights.

- **3-bit compression:** 6x memory reduction with zero accuracy loss.
- **Two-stage:** PolarQuant (MSE optimal) + QJL (Johnson-Lindenstrauss residual correction).
- **Unbiased:** Inner product estimation remains unbiased.

```
import llm.turboquant as tq

let vec = tq.vec_random(128)
let quantized = tq.quantize(vec, 3)
let reconstructed = tq.dequantize(quantized)
```

AutoResearch Agent

An autonomous research agent inspired by Karpathy's work.

- **Ratchet loop:** Propose → Train → Evaluate → Accept/Reject.
- **Multi-agent:** Collaboration between researcher, programmer, and critic agents.
- **Journal:** Keeps a detailed research log of all experiments.

Self-Evolving Architectures

`lib/llm/evolve.sage` enables models to grow their architecture over time.

- **Progressive growth:** Identity-init for depth, weight padding for width.
- **Plateau detection:** Automatically grows when training loss plateaus.

\newpage

Part IX: Appendices

\newpage

Appendix A: Built-in Functions

The following functions are available globally without any imports.

Core Functions

Function	Description
<code>print(value)</code>	Print value to stdout with newline
<code>input(prompt)</code>	Read a line from stdin
<code>len(x)</code>	Length of string, array, dict, or tuple
<code>type(x)</code>	Type name as string
<code>str(x)</code>	Convert any value to string
<code>tonumber(s)</code>	Parse string to number
<code>int(x)</code>	Truncate number to integer
<code>chr(n)</code>	Integer code point to character
<code>ord(c)</code>	Character to integer code point
<code>clock()</code>	CPU time in seconds (high resolution)
<code>hash(x)</code>	Hash value of a string
<code>doc(f)</code>	Get doc comment for a function

Array Functions

Function	Description
<code>push(arr, val)</code>	Append value to end of array
<code>append(arr, val)</code>	Alias for <code>push</code>
<code>pop(arr)</code>	Remove and return last element
<code>range(n)</code>	Array <code>[0, 1, ..., n-1]</code>
<code>range(a, b)</code>	Array <code>[a, a+1, ..., b-1]</code>
<code>slice(arr, start, end)</code>	Sub-array from start to end
<code>array_extend(a, b)</code>	Extend array a with elements of b
<code>array_repeat(a, n)</code>	Repeat array n times

String Functions

Function	Description
<code>split(str, delim)</code>	Split string by delimiter
<code>join(arr, delim)</code>	Join array elements with delimiter
<code>replace(str, old, new)</code>	Replace all occurrences
<code>upper(str)</code>	Convert to uppercase
<code>lower(str)</code>	Convert to lowercase
<code>strip(str)</code>	Trim leading/trailing whitespace
<code>startswith(str, prefix)</code>	Check if string starts with prefix
<code>endswith(str, suffix)</code>	Check if string ends with suffix
<code>contains(str, sub)</code>	Check if string contains substring
<code>indexof(str, sub)</code>	Find index of substring (-1 if not found)
<code>string_count(s, sub)</code>	Count occurrences of substring
<code>string_repeat(s, n)</code>	Repeat string n times

Dictionary Functions

Function	Description
<code>dict_keys(d)</code>	Array of all keys
<code>dict_values(d)</code>	Array of all values
<code>dict_has(d, key)</code>	Check if key exists
<code>dict_delete(d, key)</code>	Remove key from dictionary

Generator Functions

Function	Description
<code>next(gen)</code>	Advance generator and return next value

GC Functions

Function	Description
<code>gc_collect()</code>	Force a garbage collection cycle
<code>gc_stats()</code>	Return GC statistics as a dict
<code>gc_collections()</code>	Return number of GC cycles performed
<code>gc_enable()</code>	Enable the garbage collector
<code>gc_disable()</code>	Disable the garbage collector
<code>gc_set_arc()</code>	Switch to ARC mode at runtime
<code>gc_set_orc()</code>	Switch to ORC mode at runtime
<code>gc_mode()</code>	Return current GC mode string

Path Functions

Function	Description
<code>path_join(a, b)</code>	Join two path components
<code>path_dirname(path)</code>	Directory portion of path
<code>path_basename(path)</code>	Filename portion of path
<code>path_ext(path)</code>	File extension
<code>path_exists(path)</code>	Check if path exists
<code>path_is_dir(path)</code>	Check if path is a directory
<code>path_is_file(path)</code>	Check if path is a regular file
<code>path_mkpath(path)</code>	Create full directory path

VM & Gas Functions

Function	Description
<code>vm_gas_limit_set(n)</code>	Set maximum gas for VM execution
<code>vm_gas_used_get()</code>	Get amount of gas consumed

Function	Description
<code>vm_gas_limit_get()</code>	Get current gas limit

UI & Graphics Primitives

Function	Description
<code>build_quad_verts(arr)</code>	Batch generate vertices for quads
<code>build_line_quads(arr, t, ...)</code>	Convert lines to thick quads

Bytes Functions

Function	Description
<code>bytes(n)</code>	Create byte buffer of size n
<code>bytes_len(b)</code>	Length of byte buffer
<code>bytes_get(b, i)</code>	Get byte at index
<code>bytes_set(b, i, v)</code>	Set byte at index
<code>bytes_to_string(b)</code>	Convert bytes to string
<code>bytes_slice(b, start, end)</code>	Slice byte buffer
<code>bytes_push(b, v)</code>	Append byte to buffer

Memory Functions

Function	Description
<code>mem_alloc(size)</code>	Allocate raw memory
<code>mem_free(ptr)</code>	Free allocated memory
<code>mem_read(ptr, off, type)</code>	Read value at pointer offset
<code>mem_write(ptr, off, type, val)</code>	Write value at pointer offset
<code>mem_size(ptr)</code>	Size of allocation
<code>addressof(val)</code>	Get memory address of a value
<code>addressof_raw(val)</code>	Get raw memory address of a value
<code>sizeof(type)</code>	Size of a type in bytes
<code>ptr_add(ptr, off)</code>	Pointer arithmetic
<code>ptr_to_int(ptr)</code>	Convert pointer to integer

FFI Functions

Function	Description
<code>ffi_open(lib_path)</code>	Open shared library
<code>ffi_close(handle)</code>	Close shared library
<code>ffi_sym(handle, name)</code>	Look up symbol
<code>ffi_call(sym, args, ret_type)</code>	Call foreign function

Struct Interop Functions

Function	Description
<code>struct_def(fields)</code>	Define a C struct layout
<code>struct_new(def)</code>	Allocate a new struct
<code>struct_get(ptr, def, field)</code>	Read struct field
<code>struct_set(ptr, def, field, val)</code>	Write struct field
<code>struct_size(def)</code>	Size of struct in bytes

Assembly Functions

Function	Description
<code>asm_exec(code)</code>	Execute inline assembly
<code>asm_compile(code)</code>	Compile assembly to machine code
<code>asm_arch()</code>	Get current architecture string

System & SMP Functions

Function	Description
<code>cpu_count()</code>	Number of logical CPU cores
<code>cpu_physical_cores()</code>	Number of physical CPU cores
<code>cpu_has_hyperthreading()</code>	True if hyperthreading is enabled
<code>thread_set_affinity(m)</code>	Pin current thread to core mask
<code>thread_get_core()</code>	Get ID of currently executing core

Atomic Operations

Function	Description
<code>atomic_new(val)</code>	Create new atomic variable
<code>atomic_load(atom)</code>	Thread-safe load
<code>atomic_store(atom, v)</code>	Thread-safe store
<code>atomic_add(atom, v)</code>	Atomic fetch-and-add
<code>atomic_cas(a, e, d)</code>	Atomic compare-and-swap
<code>atomic_exchange(a, v)</code>	Atomic exchange

\newpage

Appendix B: CLI Reference

Usage: `sage [options] [file.sage]`

Running:

<code>(no args)</code>	Interactive REPL
<code>file.sage</code>	Run script in interpreter
<code>--repl</code>	Interactive REPL (explicit)

Compilation:

```
--compile-to-lily FILE -o OUT Emit Lily source code
--compile-from-lily FILE -o OUT Compile Lily to Sage
--emit-c FILE -o OUT Emit C source code
--compile FILE -o OUT Compile to binary via C backend
--emit-llvm FILE -o OUT Emit LLVM IR text
--compile-llvm FILE -o OUT Compile to binary via LLVM (requires clang)
--emit-asm FILE -o OUT Emit assembly source
--compile-native FILE -o OUT Compile to native binary
--compile-bare FILE -o OUT Bare-metal ELF (no libc)
--compile-uefi FILE -o OUT UEFI PE application
--emit-kotlin FILE -o OUT Emit Kotlin source code
--compile-android FILE -o DIR Generate Android Gradle project
--target ARCH Target architecture: x86-64, aarch64, rv64
```

Android Options:

```
--package PKG Android package name (default: com.sage.app)
--app-name NAME Display name for Android app
--min-sdk N Minimum Android SDK version (default: 24)
```

Runtime:

```
--runtime MODE Execution backend: ast, bytecode, jit, aot, auto
--math-work=MODES Comma-separated list of math visualization formats: grade, exec, bitwise
--jit FILE JIT compile with profiling
--aot FILE AOT compile to native binary
```

Garbage Collection:

```
--gc:tracing Tracing GC (default, concurrent mark-sweep)
--gc:arc Automatic Reference Counting
--gc:orc Optimized RC with cycle detection
```

Optimization:

```
-O0 No optimization
-O1 Constant folding
-O2 Constant folding + DCE + type specialization (Kotlin)
-O3 Aggressive (+ inlining)
-g Include debug info
```

Safety:

```
--safety Check @safe annotated functions
--strict-safety Enforce safety globally
```

Tooling:

```
--fmt FILE Format source code
--lint FILE Lint source code
--lsp Start Language Server Protocol server
fmt FILE Format source code
lint FILE Lint source code
check FILE Run type checker
safety FILE Run safety analyzer
```

Self-Hosted Compiler:

```
sage.sage FILE Run file in self-hosted interpreter
sage.sage --emit-c FILE Emit C via self-hosted compiler
sage.sage --emit-llvm FILE Emit LLVM IR via self-hosted compiler
sage.sage --emit-asm FILE Emit assembly via self-hosted compiler
sage.sage fmt FILE Format via self-hosted formatter
sage.sage lint FILE Lint via self-hosted linter
sage.sage check FILE Type check via self-hosted checker
```

REPL Commands:

```
:help Show REPL help

:env Show environment
:ast <code> Show parsed AST
:emit-c <code> Show C backend output
:emit-llvm <code> Show LLVM IR output
```

<code>:emit-kotlin <code></code>	Show Kotlin backend output
<code>:time <expr></code>	Time expression evaluation
<code>:bench N <expr></code>	Benchmark expression N times
<code>:stats</code>	Show interpreter and GC statistics
<code>:runtime MODE</code>	Switch runtime (ast, bytecode, jit, aot)

\newpage

Appendix C: Language Gotchas

This section documents known behaviors and design decisions. Items marked **FIXED** were resolved in v1.4 or later.

Still Relevant

1. **0 is truthy** -- Only false and nil are falsy. This is an intentional

design decision. Use explicit comparisons: `if x == 0: not if not x:`.

2. **chr(0) truncates strings** -- Null bytes terminate C strings in the

interpreter. The runtime uses C-style null-terminated strings, so inserting `chr(0)` into a string will truncate it at that point.

3. **No wildcard imports** -- `from module import *` is not supported. Use

`import module then module.func()`, or import specific names: `from module import func`.

4. **No multiline dict/array literals** -- Build complex structures

incrementally with assignments:

```
let config = {}
config["host"] = "localhost"
config["port"] = 8080
```

5. **match, init, and end are reserved keywords** -- **FIXED**. As of v4.1.5, `match`, `init`, `end`, `enum`, `struct`, and `trait` can be used as variable names in expression and assignment contexts, as long as they are not immediately followed by a block-starting colon or used in a declaration context.

6. **super auto-injects self** -- **FIXED**. As of v4.1.5, you can write either `super.init(args)` or `super.init(self, args)`. The interpreter and backends now handle both forms correctly.

Fixed in v1.4+

7. **~~No escape sequences~~** -- **FIXED**. `\n`, `\t`, `\r`, `\\`, `\"`,

`\xHH` and other escape sequences now work in string literals.

8. **~~No hex literals~~** -- **FIXED**. `0xFF` and `0o755` are now parsed

correctly as number literals.

9. **~~elif chains malfunction~~** -- **FIXED**. Unlimited `elif` branches

are now supported without issues.

10. **~~Instance == always false~~** -- **FIXED**. Structural equality now

works for class instances, arrays, and dicts.

11. `push/append mismatch` -- FIXED. Both `push()` and `append()`

work identically in the C interpreter. They are aliases for the same native function.

12. `% casts to int` -- FIXED. The modulo operator now uses `fmod()`

and preserves float semantics. `3.7 % 1` returns `0.7`, not `0`.

13. `Class methods can't see module-level vars` -- FIXED. Methods

now have access to their defining environment via `defining_env`.

14. `Circular Imports` -- FIXED. Circular dependencies now resolve

correctly by providing access to partially initialized state.

15. `LLVM Loop Mutation` -- FIXED. Modifying a for loop variable

inside the loop now correctly affects the iteration count in LLVM.

16. `No string interning` -- FIXED. Sage now implements global

string interning to reduce memory usage and speed up comparisons.

17. `AST Interpreter Performance` -- FIXED. The tree-walking

interpreter has been optimized with "Quickened" nodes for numeric paths.

\newpage

Appendix D: Reserved Keywords

Sage reserves the following 54 keywords. They cannot be used as variable or function names.

Control Flow

Keyword	Purpose
<code>if</code>	Conditional branch
<code>elif</code>	Else-if branch
<code>else</code>	Default branch
<code>while</code>	While loop
<code>for</code>	For-in loop
<code>in</code>	For-in iteration
<code>break</code>	Exit loop
<code>continue</code>	Skip to next iteration
<code>match</code>	Pattern matching
<code>case</code>	Match case
<code>default</code>	Match default
<code>end</code>	Block terminator

Functions and Classes

Keyword	Purpose
proc	Function definition
return	Return from function
class	Class definition
self	Instance reference
init	Constructor method
super	Parent class reference
struct	Struct definition
enum	Enum definition
trait	Trait definition

Variables

Keyword	Purpose
let	Immutable binding
var	Mutable variable

Values and Logic

Keyword	Purpose
true	Boolean true
false	Boolean false
nil	Null value
and	Logical AND
or	Logical OR
not	Logical NOT

Exceptions

Keyword	Purpose
try	Begin exception block
catch	Handle exception
finally	Always-execute block
raise	Throw exception

Advanced

Keyword	Purpose
defer	Deferred execution
yield	Generator yield
async	Async function

Keyword	Purpose
<code>await</code>	Await async result
<code>import</code>	Import module
<code>from</code>	From-import
<code>as</code>	Import alias
<code>unsafe</code>	Unsafe block
<code>print</code>	Print statement

Note: `elif` is recognized by the lexer as a combination of `else` + `if` tokens but is effectively reserved. The `end` keyword is used to terminate all block constructs: `proc`, `if`, `while`, `for`, `match`, `case`, `try`, `catch`, `finally`, `class`, `struct`, `enum`, `trait`, and `unsafe`.